

Correct and incorrect code, and “erroneous behaviour”

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Example effects](#)
4. [Motivation](#)
5. [What is code?](#)
6. [A proposal for C++](#)
7. [Concrete use case examples](#)
8. [Tooling](#)
9. [Related work](#)
10. [Implementation experience](#)
11. [Proposed wording](#)
12. [Questions and answers](#)
13. [References](#)

Revision history

- P2795R0: This initial revision.

Summary

We propose a novel kind of behaviour for C++ which allows us to formally speak about “buggy” (or “incorrect”) code, that is, code that does not mean what it should mean (in a sense we will discuss). The current C++ Standard only speaks about well-defined and well-behaved programs, and imposes no requirements on any other program. This results in an overly simple dichotomy of a program either being correct as written, with specified behaviour, or being incorrect and entirely

outside the scope of the Standard. It is not possible for program to be incorrect, yet have its behaviour constrained by the Standard.

The newly proposed *erroneous behaviour* fills this gap. It is well-defined behaviour that is nonetheless acknowledged as being “incorrect”, and thus allows implementations to offer helpful diagnostics, while at the same time being constrained by the specification.

Adopting erroneous behaviour for a particular operation consists of replacing current undefined behaviour with a (well-defined) specification of that operation’s behaviour, explicitly called out as “erroneous”. This will in general have a performance cost. *This paper does not propose any particular adoption of erroneous behaviour* (but we give examples of *possible* adoptions below). Instead, we expect each such adoption to be its own extension proposal.

The impact of changing an operation’s current undefined behaviour to erroneous behaviour is as follows:

- On correct code: none observable, but possible performance cost.
- On incorrect code: if the code leads to previously undefined behaviour that is changed to erroneous behaviour, the code is still incorrect, but the behaviour is now as specified in the chnage, not unconstrained.

Example effects

In this example we assume a hypothetical change in which the value of a default-initialized variable with automatic storage duration is erroneously well-defined. (This is a motivating use case for this proposal, but it is a strictly separate decision whether and how we would want to make such a change.)

C++20	This proposal, applied hypothetically to automatic variable initialization
<pre>extern void f(int); int x; f(x);</pre>	
undefined behaviour	erroneous behaviour
definitely a bug	definitely a bug

C++20	This proposal, applied hypothetically to automatic variable initialization
required to be accepted	may be accepted or rejected
common compilers allow rejecting (e.g. <code>-Werror</code>), this is non-conforming	conforming compilers are allowed to reject

Motivation

The pragmatic reality of real-world is that there are very few C++ programs that are entirely correct. In terms of the Standard, that means most programs are not constrained by the specification at all, since they run in to undefined behaviour. This is ultimately not very helpful to real software development efforts.

What we cannot currently do within the Standard is to talk about the behaviour of such incorrect programs. However, there has long been an active discussion around precisely this kind of behaviour: the term “safety” has been mentioned as a concern in both C and C++, but it is a nebulous and slippery term that means different things to different people. (Dave Abrahams and Sean Parent give a useful definition, see [below](#).) What makes this term hard to pin down is that we do not have an agreed upon language to define it: safety concerns the behaviour of *incorrect* programs, but incorrect programs currently aren't C++ at all! In other words, all C++ programs are safe, but most programs aren't C++ programs. Again, this is not helpful.

A simple and frequently heard suggestion is to make more code correct by changing what is currently undefined behaviour into well-defined behaviour. We would like to discuss this approach in detail.

What is code?

I would like to discuss this position, and I hope to build consensus around it.

Code is communication. Primarily, code communicates an idea *among humans*. Humans work with code as an evolving and accumulating resource. Its role in software engineering projects is not too different from the role of traditional literature in the pursuit of science, technology, and engineering: literature is how individuals learn from and contribute to collective progress. The fact that code can

also be interpreted and executed by computers is of course also important, but secondary. (There are many ways one can instruct a machine, but not all of them are suitable for building a long-term ecosystem.)

The language of code are programming languages, and the medium is source code, just like natural languages written in books, emails, or spoken in videos are the media of traditional literature. Like all media, source code is imperfect and ambiguous. The purpose of a text is to communicate an idea, but the entire communication has to be funnelled through the medium, and understood by the audience. Without the author present to explain what they really meant, the text is the only clue to the original idea; any act of reading a text is always an act of forensic reconstruction of the original idea. If the text is written well and “clear”, then readers can perform this reconstruction with high confidence that they “got it right” and feel themselves understanding the idea; they are “on the same page” as the author. On the other hand, poor writing leads to ambiguous text, and reading requires interpretation and often guess-work. This is no different in natural languages than in computer code.

I would like to propose that we appreciate the value of code *as communication with humans*, and consider how well a programming language works for that purpose in medium of source code. Source code is often shared among a large group of users, who are actively working with the code: code is only very rarely a complete black-box that can be added to a project without further thought. At the very least, interfaces and vocabulary have to be understood. But commonly, too, code has to be modified in order to be integrated into a project, and to be evolved in response to new requirements. Last but not least, code often contains errors, which have to be found, understood, and fixed. All of the above efforts may be performed by a diverse group of users, none of whom need to have intimate familiarity with any one piece of code. There is value in having *any* competent users be able to read and understand any one piece of code — not necessarily in all its domain depth, but well enough to work with it in the context of a larger project. To extent the analogy with natural language above, this is similar to how a competent speaker of a language should be able to understand and integrate a well-made argument in a discussion, even if they are not themselves an expert in the domain of the argument.

How does all this connect to C++? Like with code in any programming language, given a piece of code, a user should be able to understand the idea that the code is communicating. Absent a separate document that says “Here is what this code is meant to do:”, the main source of information available to the user is the behaviour of the code itself. Note how this has nothing to do with compiling and

running code. At this point, the code and the idea it communicates exist only in the minds of the author and the reader; no compilation is involved. How well the user understands the code depends on how ambiguous the code is, that is, how many different things it can mean. The user interprets the code by choosing a possible meaning to it from among the choices, where we assume that the code is *correct*: in C++, that means correct in the sense of the Standard, being both well-formed and executing with well-defined behaviour. This is critical: the constraint of presumed correctness serves as a dramatic aid for interpretation. If we assume that code is correct, then we can dismiss any interpretation that would require incorrect behaviour, and we only have to decide among the few remaining valid interpretations. The more valid interpretations a construction has, the more ambiguity a user has during interpretation of the entire piece of code. C++ defines only a very narrow set of behaviours, and everything else is left as the infamous *undefined behaviour*, which we could say is not C++ at all, in the sense that we assume that that's not what could possibly have been meant. Practically, of course, we would not dismiss undefined behaviour as “not C++”, but instead we would treat it as a definitive signal that the code is not communicating its idea correctly. (We could then either ask the author for clarification, or, if we are confident to have understood the correct idea anyway, we can fix the code to behave correctly. I claim that in this long-term perspective on code as a cultural good, buggy code with a clear intention is better than well-behaved, ambiguous code: if the intention is clear, then I can see if the code is doing the right thing and fix it if not, but without knowing the intention, I have no idea if the well-behaved code is doing what it is supposed to.)

A proposal for C++

We can finally state a current problem in C++ and propose a solution. Current C++ has no formal concept of incorrect code beyond undefined behaviour (which includes any execution of an ill-formed program). This would be sufficient in a world where C++ programs were generally correct, but in practice, the complexity both inherent to the language and inevitable in projects of any significant size means that real C++ programs are almost never free from errors. The tension arises because such programs are deployed in the real world, but their behaviour is entirely unconstrained by the C++ standard, and in practice there have been a significant number of dangerous or outright harmful behaviours. [References to CVEs etc.] From the point of view of actual *behaviour*, as opposed to the intended *meaning* of code discussed in the previous section, this situation is problematic, since the very real danger and harm are immediate problems.

Our proposed solution is to address both actual behaviour and the meaning of “code as a medium” by formally acknowledging incorrect behaviour, and allowing the language specification to take it into account:

- We want the actual behaviour of code to fall under the scope of the Standard (different from today).
- We want to retain the power of code to communicate ideas, which concretely means that we do not want to make previously undefined behaviour assume meaning. (We can make it well-defined, but we want to preserve the fact that it can never have been intentional on part of the author.)

The proposal is to add a novel kind of behaviour to C++, which we tentatively call *erroneous behaviour*:

erroneous behaviour: well-defined behaviour (which includes implementation-defined and unspecified behaviour) which allows the implementation to issue a diagnostic message

Erroneous behaviour is never intended. Its presence is therefore a definitive sign that the code is not communicating correctly and needs to be fixed. At the same time, erroneous behaviour is indeed defined. Implementations must exhibit the defined behaviour, at least up until a diagnostic is issued (if ever). There is no risk of damage or harm from executing erroneous behaviour.

Erroneous behaviour can be used to improve safety if we use it to undefined behaviour from constructs that are currently frequently found in incorrect code. This would change such code to be correct, though unintended/erroneous. Its behaviour would no longer be unconstrained, but instead be specified by the Standard. At the same time, readers can now assume that C++ code is both correct and not erroneous, and thus continue to be able to interpret code in the same way as they do today, without having to assume that code could suddenly acquire new meaning.

Note that the diagnostic is not immediately tied to the erroneous behaviour but could be issued at any later point. Implementations that want to diagnose would not be required to track every behaviour immediately, but could, for example, aggregate findings and only diagnose some sparse checkpoints.

Once the notion of erroneous behaviour exists in the Standard, we can entertain proposals to replace current undefined behaviour with erroneous behaviour. This generally comes at a performance cost, and should be a case-by-case analysis. The difference between undefined and erroneous behaviours is that the latter constrains implementations, and we need to consider whether that is a worthwhile

trade-off, considering the likelihood of user mistakes and the the potential danger of undefined behaviour. We will give example applications in the next section, but they do not form part of this proposal.

Concrete use case examples

Overview

This section shows a few possible applications of erroneous behaviour. That is, each example describes an operation that currently has undefined behaviour, and suggests how one might define the behaviour, and make it erroneous.

Initialization automatic variables

[to be fleshed out] Proposal: make it so that `int x;` gives `x` an implementation-defined object representation and an indeterminate value representation, and if glvalue-to-prvalue conversion is applied to such a glvalue of indeterminate value, the result is the value implied by that object representation, and the behaviour is erroneous.

Note that we do not actually make the value 0, but leave it up to the implementation. The assumption is that production implementations will use 0 and never diagnose the erroneous behaviour, whereas debug builds might put some other pattern into the initial value, a hardened build might select a runtime-randomized value (to detect unwarranted reliance on the value), and runtime sanitizers can track access to the uninitialized value and diagnose it. This is ultimately a detail, but we consider this choice superior to guaranteeing the value zero.

Type punning

Proposal: make type-punned access erroneous, so as to address this related vulnerability:

```
float x;  
print_secret(reinterpret_cast<int&>(x));
```

This might be achieved by erroneously giving the access the value that is assumed by the object representation, or some other value, or erroneously terminate.

Note that the proposal from P2723R1 to make automatic-storage variables initialized to zero does not actually help in this example, since the compiler can still see the undefined behaviour from the type-punned access and thus is permitted to perform the initialization.

Signed integer overflow

Proposal: make the value of signed integer overflow to erroneously be the twos'-complement value.

This proposal is perhaps special in that it does not need to incur a direct implementation cost on platforms that already perform this operation in hardware. (However, the cost of the missed optimisation based on reaching undefined behaviour remains.)

Pointer arithmetic, array access

Proposal: dereferencing a null pointer could result, erroneously, in some fixed value, or terminate erroneously. Similarly for accessing an array of known bound out of bounds.

This clearly has a runtime cost. For arrays this would be mandatory bounds checking for all arrays of known bound.

User-defined erroneous behaviour

Proposal: define a magic library function:

```
std::erroneous_behaviour();
```

Effects: Does nothing, erroneously.

Thus function can allows library code to define its own erroneous behaviour, and reaching this function could be detectable by tooling.

Preconditions in the standard library

The standard library currently imposes *Preconditions*: on functions, and violation of those preconditions is defined to result in undefined behaviour. This is sometimes called "soft" or "library" undefined behaviour, since it is not actually detectable in the core language, but in general only by a human reader.

We could add a variation of preconditions that lead to erroneous behaviour. For those, we would have to specify the outcome, but reaching such behaviour could be detectable by tooling.

API design choices

This is an example of the various options for putting contracts (both explicit and implicit) on a function API. Consider a hypothetical array with some knowable bound, and an accessor function that we will discuss:

```
extern int data[];    // size given by data_len
extern int data_len;

int get_data(std::size_t i);
```

- **Narrow contract, with UB:** “Returns `data[i]`, requires `0 <= i && i < 0.`” Implementation:

```
return data[i];
```

This is “unsafe” in the sense of Abrahams and Parent.

- **Narrow contract, preconditions asserted:** “Returns `data[i]`, requires `0 <= i && i < data_len.`” Implementation:

```
if (0 <= i && i < data_len) return data[i];
std::abort();
```

This is “strongly safe” in the sense of Abrahams and Parent. Human readers can see that violating preconditions is an error, but it is hard for tooling to see this.

- **Narrow contract, erroneous result:** “Returns `data[i]` if `0 <= i && i < data_len`, otherwise erroneously returns `-8.`” Implementation:

```
if (0 <= i && i < data_len) return data[i];
std::erroneous_behaviour();
return -8;
```

This has no undefined behaviour, but detectable erroneous behaviour.

- **Wide contract, preconditions asserted:** “Returns `data[i]` if `0 <= i && i < data_len`, otherwise terminates.” Implementation:

```
if (0 <= i && i < data_len) return data[i];
std::abort();
```

Similar to “narrow contract, preconditions asserted”, but here termination is part of the API. E.g. users could call this function for the deliberate purpose of terminating, and that would not be a programming error.

- **Wide contract:** “Returns `data[i]` if `0 <= i && i < data_len`, otherwise returns `-8`.” Implementation:

```
if (0 <= i && i < data_len) return data[i];
return -8;
```

Tooling

While we have been emphasising the importance of code readability and understandability, we must also consider the practicalities of actually compiling and running code. Whether code has meaning, and if so, which, impacts tools. There are two important, and sometimes opposed, use cases we would like to consider.

Production compilers

Getting code to run in production often comes with two important (and also opposed) expectations:

- Performance: code should use as few resources as possible to achieve its specified behaviour.
- Safety: incorrect code should not have harmful side effects.

Undefined behaviour, and in particular its implications on the meaning of code, is increasingly exploited by compilers to optimize code generation. By assuming that undefined behaviour can never have been intentional, transitive assumptions can be derived that allow for far-reaching optimizations. This is often desirable and beneficial for correct code (and demonstrates the value of unambiguously understandable code: even compilers can use this reasoning to determine how much work does and does not have to be done). However, for incorrect code this can expose vulnerabilities, and thus constitute a considerable lack of safety. [P1093R0](#) discusses these performance implications of undefined behaviour.

The proposed erroneous behaviour retains the same meaning of code as undefined behaviour for human readers, but the compiler has to accept that erroneous

behaviour can happen. This constrains the compiler (as it has to ensure erroneous results are produced correctly), but in the event of incorrect code (which all erroneous behaviour requires), the resulting behaviour is constrained by the Standard and does not create a safety hazard. In other words, erroneous behaviour has a potential performance cost compared to undefined behaviour, but is safer in the presence of incorrect code.

Debug toolchains and sanitizers

The other major set of tools that software projects use are debugging tools. Those include extra warnings on compilers, static analysers, and runtime sanitizers. The former two are good at catching some localised bugs early, but do not catch every bug. Indeed one of the main limitations we seem to be discovering is that there is reasonable C++ code for which important analyses cannot be performed statically. (Note that [P2687R0](#) proposes a safety strategy in which static analysis plays a major role.) Runtime sanitizers like ASAN, MSAN, TSAN, UBSAN, on the other hand, have excellent abilities to detect undefined behaviour at runtime with virtually no false positives, but at a significant build and runtime cost.

Both runtime sanitizers and static analysis can use the code readability signal from both undefined and erroneous behaviour equally well. In both cases it is clear that the code is incorrect. For undefined behaviour, implementations are unconstrained anyway and tools may reject or diagnose at runtime. The goal of erroneous behaviour is to permit the exact same treatment, by allowing a conforming implementation to diagnose, terminate (and also reject) a program that contains erroneous behaviour.

In other words, erroneous behaviour retains the understandability and debuggability of undefined behaviour, but also constrains the implementation just like well-defined behaviour.

Usage profiles

The following toolchain deployment examples are based on real-world setups.

- A safety-critical production system (e.g. one that handles user data) compiles code treating erroneous behaviour as well-defined, not issuing diagnostics. Compared to the status quo, the resulting executable is safer since erroneous behaviour is well-defined.
- A safety-noncritical, high-performance system (e.g. a scientific simulation) may use a toolchain that assumes that no erroneous behaviour exists (similar to `-ffast-math`, which assumes that all floating point values lie in a certain

range). This allows for best-possible performance, and corresponds to the status quo. Debugging and testing are performed separately.

- Debugging and testing deployments, including unit tests, use runtime sanitizers to detect undefined and erroneous behaviour. This is a continuous part of a larger production ecosystem, where test coverage and fuzzing tools try to expose as much of the production code to sanitizers as possible, and it is cultivated as part of the overall production effort.

Related work

Sean Parent's presentation [Reasoning About Software Correctness](#) (and also subsequent Cpp North 2022 keynote talk) give a useful definition of "safety" adapted specifically to C++ and which explicitly only concerns *incorrect code*. He defines a function to be safe if it does not lead to undefined behaviour (that is, even when preconditions are violated). He points out that safety composes, whereas correctness does not (i.e. a composition of safe calls is itself safe, whereas a composition of correct calls is not intrinsically correct, but only if preconditions hold). He argues, similarly to our argument in this proposal, that it would not be helpful to turn undefined behaviour into well-defined behaviour for the purpose of making functions safe, since that would merely hide bugs; instead, a helpful change would be for unsafe operation to become "strongly safe", which for him means that it terminates on precondition violation. If the current proposal were to adopt these terms, we would instead merely require that strong safety would lead to erroneous behaviour (rather than outright termination), and leave the actual behaviour implementation-defined.

JF Bastien's paper P2723R1 proposes addressing the safety concerns around automatic variable initialization by just defining variables to be initialized to zero. The previous revision of that paper was what motivated the current proposal: The resulting behaviour is desirable, but the cost on code understandability is unacceptable to the present author.

The papers P2687R0 by Bjarne Stroustrup and Gabriel Dos Reis and P2410R0 by Bjarne take a more general look at how to arrive at a safe language. They recommends a combination of static analysis and restrictions on the use of the language so as to make static analysis very effective. However, on the subject of automatic variable initialization specifically they offers no new solution: P2687R0 only recommends either zero-initialization or annotated non-initialization (reading of which results in UB); in that regard it is similar to JF Bastien's proposal. P2410R0 states that "[s]tatic analysis easily prevents the creation of uninitialized

objects”, but the intended result of this prevention, and in particular the impact on code understandability, is left open.

Tom Honerman proposed a system of “diagnosable events”, which is largely aligned with the values and goals of this proposal, and takes a quite similar approach: Diagnosable events have well-defined behaviour, but implementations are permitted to handle them in an implementation-defined way.

Davis Herring's paper P1492R2 proposes a checkpointing system that would stop undefined behaviour from having arbitrarily far-reaching effects. That is a somewhat different problem area from the present safety one, and in particular, it does not control the effects of the undefined behaviour itself, but merely prevents it from interfering with other, previous behaviour. (E.g. this would not prevent the leaking of secrets via uninitialized variables.)

The Ada programming language has a notion of bounded undefined behaviour.

Paper P1093R0 by Bennieston, Coe, Gahir and Russel discusses the value of undefined behaviour in *correct* code and argues for the value of the compiler optimizations that undefined behaviour permits. This is essentially the tool's perspective of the value of undefined behaviour for the interpretability of code which we discussed above: both humans and compilers benefit from being able to understand code with fewer ambiguities. Compilers can use the absence of ambiguities to avoid generating unnecessary code. The paper argues that we should not break these optimizations lightheartedly by making erstwhile undefined behaviour well-defined.

Implementation experience

Applying erroneous behaviour to the default initialization of automatic variables is already available today. Clang and GCC expose an example of the new production behaviour when given the flag `-ftrivial-auto-var-init=zero`. Clang exposes the error-detecting behaviour when using its Memory Sanitizer (which currently detects undefined behaviour, and would have to be taught to also recognize erroneous behaviour).

The proposal primarily constitutes a change of the specification tools that we have available in the Standard, so that we have a formal concept of incorrect code that the Standard itself can talk about. It should only pose a minor implementation burden.

Proposed wording

Add an entry to [3, intro.defs]:

3.? erroneous behaviour [defns.erroneous]

well-defined behavior (including implementation-defined and unspecified behavior) which is subject to additional conformance constraints

[Note 1 to entry: Erroneous behaviour is always the consequence of incorrect program code. Implementations are allowed, but not required to diagnose it ([4.1.1, intro.compliance.general]). — end note]

Modify and add a list item to [4.1.1, intro.compliance.general] paragraph 2:

- If a program contains no violations of the rules in [Clause 5, lex] through [Clause 33] and [Annex D, depr], a conforming implementation shall, within its resource limits as described in [Annex B, implimits], accept and ~~correctly~~-execute[footnote] that program as defined.
- If a program is as in the previous sentence except that it also contains an occurrence of a construct described in this document as having “erroneous behaviour”, a conforming implementation shall also accept and execute that program as above, except that it is allowed to issue a diagnostic when or after the erroneous behavior is executed, and after such a diagnostic the behaviour is implementation-defined. Additionally, the implementation is not required to accept a translation unit ([lex.separate, 5.1]) if it can determine that erroneous behaviour is reachable within that translation unit. [Note: It is intended that an implementation either execute erroneous behavior without diagnostic, or issue a diagnostic and continue, or issue a diagnostic and terminate. —end note]
- If a program contains a violation of any diagnosable rule or an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct, a conforming implementation shall issue at least one diagnostic message.
- ...

Questions and answers

Do you really mean that there can never be any UB in any correct code?

There is of course always room for nuance and detail. If a particular construction is known to be UB, but still appropriate on some platform or under some

additional assumptions, it is perfectly fine to use it. It should be documented/annotated sufficiently, and perhaps tools that detect UB need to be informed that the construction is intentional.

Why is static analysis not enough to solve the safety problem of UB? Why do we need sanitizers? Current C++ is not constrained enough to allow static analysis to accurately detect all cases of undefined behaviour. (For example, C++ allows initializing a variable via a call to a function in a separate translation unit or library.) Other languages like Rust manage to prevent unsafe behaviour statically, but they are more constrained (e.g. Rust does not allow passing an uninitialized value to a function). Better static analysis is frequently suggested as a way to address safety concerns in C++ (e.g. P2410R0, P2687R0), but this usually requires adopting a limited subset of C++ that is amenable to reliable static analysis. This does not help with the wealth of existing C++ code, neither with making it safe nor with making it correct. By contrast, runtime sanitizers can reliably point out when undefined behaviour is reached.

Why is `int x;` any different from `std::vector<int> v;`? Several reasons. One is that `vector` is a class with internal invariants that needs to be destructible, so a well-defined initial state already suggests itself. The other is that a vector is a container of elements, and if the initializer does not provide any elements, then a vector with no elements is an unsurprising result. By contrast, if there is no initial value given for an `int`, there is no single number that is better or more obviously right than any other number. Zero is a common choice in other languages, but it does not seem helpful in the sense of making it easy to write unambiguous code if we allow a novel spelling of a zero-valued `int`. If you mean zero, just say `int x = 0;`.

Is this proposal better than defining `int x;` to be zero? It depends on whether you want code to deliberately use `int x;` to mean, deliberately, that `x` is zero. The counter-position, shared by this author, is that zero should have no such special treatment, and all initialization should be explicit, `int x = -1, y = 0, z = +1;`. All numeric constants are worth seeing explicitly in code, and there is no reason to allow `int x;` as a valid alternative for one particular case that already has a perfectly readable spelling. (An explicit marker for a deliberately uninitialized variable is still a good idea, and accessing such a variable would remain undefined behaviour, and not become erroneous even in this present proposal.)

References

- Andrew Bennieston, Jonathan Coe, Daven Gahir, Thomas Russel, [P1093R0](#): *Is undefined behaviour preserved?*
- Tom Honerman, *Posioned values: A feature and specification mechanism to aid diagnosis of implicitly initialized variables*, private communication.
- Davis Herring, [P1494R2](#): *Partial program correctness*.
- JF Bastien, [P2723R1](#): *Zero-initialize objects of automatic storage duration*.
- Dave Abrahams, private communication.
- Sean Parent, [Reasoning About Software Correctness](#).
- Sean Parent, Cpp North 2022 keynote talk, [The Tragedy of C++](#).
- Bjarne Stroustrup, Gabriel Dos Reis, [P2687R0](#): *Design Alternatives for Type-and-Resource Safe C++*.
- Bjarne Stroustrup, [P2410R0](#): *Type-and-resource safety in modern C++*.